```python
import os
import shutil

# Created the 'data' directory
os.makedirs('/content/data', exist_ok=True)

# Define the full path for the CSV file
csv_file_path = '/content/data/source.csv'

# Checking if the path exists
if os.path.exists(csv_file_path) and os.path.isdir(csv_file_path):
    shutil.rmtree(csv_file_path)
    print(f"Removed conflicting directory: {csv_file_path}")

# Write the CSV content to a file
csv_content =
"""product_id,old_name,category,price,status,amount,base_price,region,
priority,user_id
P101,Mobile,Electronics,25000,Completed,1200,25000,North,High,U101
P102,Shirt,Clothing,1500,Pending,800,1500,South,Low,U102
P103,Laptop,Electronics,55000,Completed,2500,55000,East,High,U103
P104,Table,Furniture,7000,Completed,1500,7000,North,Medium,
P105,Headphones,Electronics,3000,Cancelled,900,3000,West,Low,U105"""

with open(csv_file_path, 'w') as f:
    f.write(csv_content)

print(f"'{csv_file_path}' created successfully.")
```

```
'/content/data/source.csv' created successfully.
```

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
import os

spark = SparkSession.builder \
    .appName("Week6 Spark Assignment") \
    .getOrCreate()

# Reading CSV file
df = spark.read.csv(
    "/content/data/source.csv",
    header=True,
    inferSchema=True
)

print("Original Data:")
df.show(5)

print("Schema:")
```

```
df.printSchema()


Original Data:
+----------+----------+-----------+-----+---------+------+----------
+------+--------+-------+
|product_id|  old_name|   category|price|   status|amount|base_price|
region|priority|user_id|
+----------+----------+-----------+-----+---------+------+----------
+------+--------+-------+
|      P101|    Mobile|Electronics|25000|Completed|  1200|     25000|
North|    High|   U101|
|      P102|     Shirt|   Clothing| 1500|  Pending|   800|      1500|
South|     Low|   U102|
|      P103|    Laptop|Electronics|55000|Completed|  2500|     55000|
East|    High|   U103|
|      P104|     Table|  Furniture| 7000|Completed|  1500|      7000|
North|  Medium|   NULL|
|      P105|Headphones|Electronics| 3000|Cancelled|   900|      3000|
West|     Low|   U105|
+----------+----------+-----------+-----+---------+------+----------
+------+--------+-------+

Schema:
root
 |-- product_id: string (nullable = true)
 |-- old_name: string (nullable = true)
 |-- category: string (nullable = true)
 |-- price: integer (nullable = true)
 |-- status: string (nullable = true)
 |-- amount: integer (nullable = true)
 |-- base_price: integer (nullable = true)
 |-- region: string (nullable = true)
 |-- priority: string (nullable = true)
 |-- user_id: string (nullable = true)


# Selecting required columns and filtering Electronics category
electronics_df = df.filter(col("category") == "Electronics") \
                   .select("product_id", "price")

print("Electronics Products:")
electronics_df.show(5)

# Renaming column and casting price to double
# 'product_id' renamed to 'item_id'
revised_df = df.withColumnRenamed("product_id", "item_id") \
               .withColumn("price", col("price").cast("double"))

print("Revised DataFrame:")
```

```
revised_df.printSchema()
revised_df.show(5)


Electronics Products:
+----------+-----+
|product_id|price|
+----------+-----+
|      P101|25000|
|      P103|55000|
|      P105| 3000|
+----------+-----+

Revised DataFrame:
root
 |-- item_id: string (nullable = true)
 |-- old_name: string (nullable = true)
 |-- category: string (nullable = true)
 |-- price: double (nullable = true)
 |-- status: string (nullable = true)
 |-- amount: integer (nullable = true)
 |-- base_price: integer (nullable = true)
 |-- region: string (nullable = true)
 |-- priority: string (nullable = true)
 |-- user_id: string (nullable = true)

+-------+----------+-----------+-------+---------+------+----------
+------+--------+-------+
|item_id|  old_name|   category|  price|   status|amount|base_price|
region|priority|user_id|
+-------+----------+-----------+-------+---------+------+----------
+------+--------+-------+
|   P101|    Mobile|Electronics|25000.0|Completed|  1200|     25000|
North|    High|   U101|
|   P102|     Shirt|   Clothing| 1500.0|  Pending|   800|      1500|
South|     Low|   U102|
|   P103|    Laptop|Electronics|55000.0|Completed|  2500|     55000|
East|    High|   U103|
|   P104|     Table|  Furniture| 7000.0|Completed|  1500|      7000|
North|  Medium|   NULL|
|   P105|Headphones|Electronics| 3000.0|Cancelled|   900|      3000|
West|     Low|   U105|
+-------+----------+-----------+-------+---------+------+----------
+------+--------+-------+


# Filtering completed orders with amount greater than 1000
completed_orders = df.filter(
    (col("status") == "Completed") & (col("amount") > 1000)
)
```

```
print("Completed Orders with Amount Greater Than 1000:")
completed_orders.show(5)
```

```
Completed Orders with Amount Greater Than 1000:
+----------+--------+-----------+-----+---------+------+----------
+------+--------+-------+
|product_id|old_name|   category|price|   status|amount|base_price|
region|priority|user_id|
+----------+--------+-----------+-----+---------+------+----------
+------+--------+-------+
|      P101|  Mobile|Electronics|25000|Completed|  1200|     25000|
North|    High|   U101|
|      P103|  Laptop|Electronics|55000|Completed|  2500|     55000|
East|    High|   U103|
|      P104|   Table|  Furniture| 7000|Completed|  1500|      7000|
North|  Medium|   NULL|
+----------+--------+-----------+-----+---------+------+----------
+------+--------+-------+
```

```
# Adding final price column with 18% tax
final_price_df = df.withColumn(
    "final_price",
    col("base_price") * 1.18
)

print("Data with Final Price:")
final_price_df.show(5)
```

```
Data with Final Price:
+----------+----------+-----------+-----+---------+------+----------
+------+--------+-------+-----------+
|product_id|  old_name|   category|price|   status|amount|base_price|
region|priority|user_id|final_price|
+----------+----------+-----------+-----+---------+------+----------
+------+--------+-------+-----------+
|      P101|    Mobile|Electronics|25000|Completed|  1200|     25000|
North|    High|   U101|    29500.0|
|      P102|     Shirt|   Clothing| 1500|  Pending|   800|      1500|
South|     Low|   U102|     1770.0|
|      P103|    Laptop|Electronics|55000|Completed|  2500|     55000|
East|    High|   U103|    64900.0|
|      P104|     Table|  Furniture| 7000|Completed|  1500|      7000|
North|  Medium|   NULL|     8260.0|
|      P105|Headphones|Electronics| 3000|Cancelled|   900|      3000|
West|     Low|   U105|     3540.0|
+----------+----------+-----------+-----+---------+------+----------
```

```
+------+--------+-------+-----------+


# Filtering North region or High priority
region_priority_df = df.filter(
    (col("region") == "North") | (col("priority") == "High")
)

print("North Region or High Priority Data:")
region_priority_df.show(5)


North Region or High Priority Data:
+----------+--------+-----------+-----+---------+------+----------
+------+--------+-------+
|product_id|old_name|   category|price|   status|amount|base_price|
region|priority|user_id|
+----------+--------+-----------+-----+---------+------+----------
+------+--------+-------+
|      P101|  Mobile|Electronics|25000|Completed|  1200|     25000|
North|    High|   U101|
|      P103|  Laptop|Electronics|55000|Completed|  2500|     55000|
East|    High|   U103|
|      P104|   Table|  Furniture| 7000|Completed|  1500|      7000|
North|  Medium|   NULL|
+----------+--------+-----------+-----+---------+------+----------
+------+--------+-------+


# -- Created a dummy Parquet file for demonstration -- #
# This step is added to ensure the subsequent parquet read operation
has a valid source.
# In a real scenario, parquet_df would be read from an existing file.
output_parquet_path = "/content/output/parquet_data"
os.makedirs(output_parquet_path, exist_ok=True)

# Create a dummy DataFrame to write as Parquet
dummy_data_for_parquet = [
    ("U101", "productA", 100),
    ("U102", "productB", 200),
    (None, "productC", 50),
    ("U104", "productD", 150)
]
dummy_parquet_df = spark.createDataFrame(dummy_data_for_parquet,
["user_id", "product", "value"])
dummy_parquet_df.write.mode("overwrite").parquet(output_parquet_path)

print(f"Dummy Parquet file created at {output_parquet_path}")

# Reading Parquet file and removing rows where user_id is null
parquet_df = spark.read.parquet(output_parquet_path)
```

```
cleaned_users = parquet_df.filter(col("user_id").isNotNull())
```

```
Dummy Parquet file created at /content/output/parquet_data
```

```
# Defining output CSV path
output_csv_path = "/content/output/cleaned_users_csv"
os.makedirs(output_csv_path, exist_ok=True)

cleaned_users.write.mode("overwrite") \
                   .option("header", True) \
                   .csv(output_csv_path)

print(f"Parquet data filtered and saved as CSV successfully to
{output_csv_path}.")
```

```
Parquet data filtered and saved as CSV successfully to
/content/output/cleaned_users_csv.
```